

TEMA 1 · ENTORNOS DE DESARROLLO

Desarrollo de software

Tipos de software · Fases del desarrollo · Ciclo de vida · Lenguajes de programación ·
Frameworks y entornos de ejecución



Vas a saber...

01

Distinguir los tipos de software

De sistema, de desarrollo y de aplicación

02

Recorrer las fases del desarrollo

Del análisis al mantenimiento

03

Elegir un modelo de ciclo de vida

Cascada, realimentación y modelos evolutivos

04

Clasificar lenguajes y herramientas

Alto nivel, POO, frameworks y entornos de ejecución

El tema en cinco bloques

1

El software y el hardware

Tipos de software y su relación con la máquina

2

El desarrollo del software

Las siete fases, del análisis al mantenimiento

3

El ciclo de vida del software

Cascada, realimentación y modelos evolutivos

4

Lenguajes de programación

Evolución, concepto y técnicas de programación

5

Herramientas de apoyo

Frameworks y entornos de ejecución

¿Qué es el software?

Conjunto de programas informáticos que actúan sobre el hardware para ejecutar lo que el usuario desee

Según su función se distinguen tres tipos de software:

DE SISTEMA

Su objetivo es liberar al usuario de los detalles del hardware que se usa y de su gestión.

DE DESARROLLO

Su objetivo es proporcionar herramientas al usuario para el desarrollo de programas informáticos.

DE APLICACIÓN

Su objetivo es permitir al usuario realizar una o varias tareas específicas.

El ordenador se compone de dos partes bien diferenciadas: hardware y software.

Qué incluye cada tipo

SIS

De sistema

Sistema operativo

Controladores de dispositivos (drivers)

Utilidades

El sistema operativo es el software base que ha de estar instalado y configurado para que las aplicaciones puedan ejecutarse.

Ejemplos: Windows, Linux, Mac OS X.

DES

De desarrollo

Editores de texto

Compiladores

Intérpretes

Enlazadores

Depuradores

Entornos integrados de desarrollo (IDE)

APL

De aplicación

Aplicaciones ofimáticas

Software educativo

Bases de datos

Videojuegos

Software de diseño asistido (CAD)

...

El software de desarrollo es, precisamente, el objeto de este módulo.

Relación hardware-software

Existe una relación indisoluble entre ambos: necesitan estar instalados y configurados correctamente para que el equipo funcione. Podemos ponerla de manifiesto desde dos puntos de vista.

SO

Desde el sistema operativo

El sistema operativo coordina al hardware durante el funcionamiento del ordenador, actuando como intermediario entre éste y las aplicaciones que están corriendo.

Todas las aplicaciones necesitan recursos hardware: tiempo de CPU, espacio en memoria RAM, tratamiento de interrupciones, gestión de los dispositivos de E/S.

El sistema operativo controla todos estos aspectos de manera «oculta» para las aplicaciones y para el usuario.

APP

Desde las aplicaciones

Una aplicación es un conjunto de programas escritos en algún lenguaje de programación que el hardware debe interpretar y ejecutar.

Todos los lenguajes están escritos con sentencias de un idioma que el ser humano puede aprender y usar fácilmente.

El hardware, en cambio, sólo es capaz de interpretar señales eléctricas: ausencias o presencias de tensión.

El problema de la traducción

¿Cómo será capaz el ordenador de «entender» algo escrito en un lenguaje que no es el suyo?

El programador escribe

Sentencias de un idioma humano

El hardware entiende

0 y 1 (código binario)

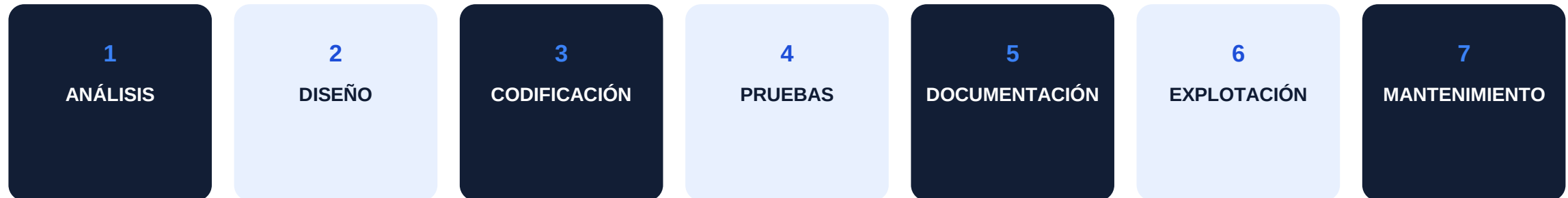
Tendrá que pasar algo: un proceso de traducción de código

El hardware sólo interpreta señales eléctricas —ausencias o presencias de tensión— que en informática se traducen en secuencias de 0 y 1. Ese proceso de traducción es el que permite al ordenador ejecutar las instrucciones escritas en un lenguaje de programación.

¿Qué es el desarrollo de software?

Todo el proceso que ocurre desde que se concibe una idea hasta que un programa está implementado en el ordenador y funcionando

Consta de una serie de pasos de obligado cumplimiento: sólo así podremos garantizar que los programas creados son eficientes, fiables, seguros y responden a las necesidades de los usuarios finales.



Las siete fases principales del desarrollo del software.

Análisis

Se determinan y definen claramente las necesidades del cliente y se especifican los requisitos que debe cumplir el software a desarrollar

La fase de mayor importancia

Todo lo demás dependerá de lo bien detallada que esté.

También la más complicada

No está automatizada y depende en gran medida del analista que la realice.

La culminación de esta fase es el documento ERS (Especificación de Requisitos Software).

La especificación de requisitos debe...

Ser completa y sin omisiones

Ser concisa y sin trivialidades

Evitar ambigüedades. Utilizar lenguaje formal

Evitar detalles de diseño o implementación

Ser entendible por el cliente

Separar requisitos funcionales y no funcionales

Dividir y jerarquizar el modelo

Fijar criterios de validación

Requisitos funcionales: qué funciones tendrá que realizar la aplicación. · **Requisitos no funcionales:** tiempo de respuesta, tratamiento ante peticiones simultáneas...

El documento ERS

Especificación de Requisitos Software. En este documento quedan especificados:

1

La planificación de las reuniones que van a tener lugar

2

Relación de los objetivos del usuario cliente y del sistema

3

Relación de los requisitos funcionales y no funcionales del sistema

4

Relación de objetivos prioritarios y temporización

5

Reconocimiento de requisitos mal planteados o que conllevan contradicciones, etc.

Diseño

Se divide el sistema en partes, se establecen las relaciones entre ellas y se decide qué hará exactamente cada parte

En esta fase se deben tomar decisiones importantes, tales como:

Entidades y relaciones

de las bases de datos

Lenguaje de programación

que se va a utilizar

Sistema Gestor de

Bases de Datos

...

otras decisiones de diseño

La selección del lenguaje de programación es una decisión de diseño, no de codificación.

Codificación

Se elige un Lenguaje de Programación y se codifican los programas. La tarea la realiza el programador y tiene que cumplir exhaustivamente con todos los datos impuestos en el análisis y en el diseño.

1

Modularidad

Que esté dividido en trozos más pequeños

2

Corrección

Que haga lo que se le pide realmente

3

Fácil de leer

Para facilitar su desarrollo y mantenimiento futuro

4

Eficiencia

Que haga un buen uso de los recursos

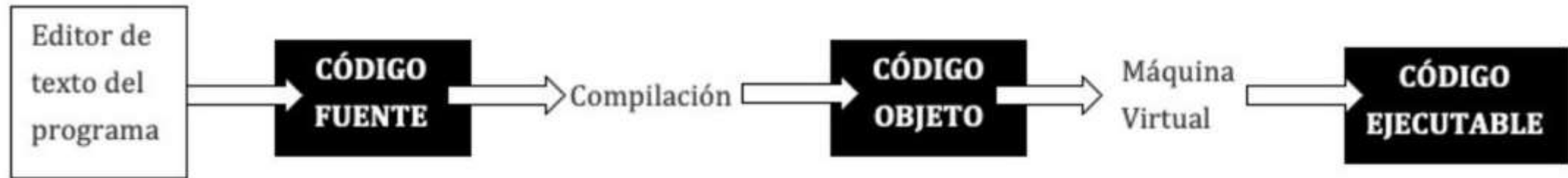
5

Portabilidad

Que se pueda implementar en cualquier equipo

Características deseables de todo código.

Los tres estados del código



Código fuente

Escrito por los programadores en un editor de texto, con un lenguaje de alto nivel

Código objeto

Código binario intermedio, resultado de compilar el código fuente

Código ejecutable

Código binario directamente inteligible por la computadora

Esquema del material del módulo: del editor de texto al código ejecutable.

Código fuente y algoritmo

Código fuente

Es el escrito por los programadores en algún editor de texto. Se escribe usando algún lenguaje de programación de alto nivel y contiene el conjunto de instrucciones necesarias.

Algoritmo

Conjunto de pasos a seguir para obtener la solución del problema. Su elaboración previa es un aspecto muy importante de esta fase.

El algoritmo lo diseñamos en pseudocódigo: con él, la codificación posterior a un Lenguaje de Programación será más rápida y directa

Primero se piensa la solución, después se escribe en un lenguaje concreto.

Código objeto: compilar o interpretar

Qué es el código objeto

Es un código intermedio: el resultado de traducir código fuente a un código equivalente formado por unos y ceros que aún no puede ser ejecutado directamente por la computadora.

Sólo se genera una vez que el código fuente está libre de errores sintácticos y semánticos.

Compilación

Traducción de una sola vez del programa, realizada con un compilador.

El código objeto sólo existe si el programa se compila.

Interpretación: traducción y ejecución simultánea del programa, línea a línea. Los programas interpretados NO producen código objeto; el paso de fuente a ejecutable es directo. El software responsable se llama intérprete.

Dos formas de traducir el código fuente a código ejecutable.

Código ejecutable y máquina virtual

Código ejecutable

Código binario resultante de enlazar los archivos de código objeto con ciertas rutinas y bibliotecas necesarias, a través de la máquina virtual.

También se conoce como código máquina y ya sí es directamente inteligible por la computadora.

Consta de un único archivo que puede ser directamente ejecutado. No necesita ninguna aplicación externa: lo ejecuta y controla el sistema operativo.

Máquina virtual

Tipo especial de software cuya misión es separar el funcionamiento del ordenador de los componentes hardware instalados.

Permite desarrollar y ejecutar una aplicación sobre cualquier equipo, con independencia de las características concretas de los componentes físicos instalados.

Esto es lo que garantiza la portabilidad de las aplicaciones

Pruebas

Se prueban los programas para detectar errores y se depuran. Se realizan sobre un conjunto de datos de prueba: un conjunto seleccionado y predefinido de datos límite a los que la aplicación es sometida.

1

Pruebas unitarias

Probar, una a una, las diferentes partes del software y comprobar su funcionamiento por separado, de manera independiente.

2

Pruebas de integración

Se realizan una vez superadas las unitarias. Comprueban el funcionamiento del sistema completo, con todas sus partes interrelacionadas.

La prueba final se denomina Beta Test: se realiza sobre el entorno de producción donde el software va a ser utilizado por el cliente. El período de prueba será el pactado con el cliente.

Documentación: tres grandes documentos

GT

Guía Técnica

Objetivo: facilitar un correcto desarrollo, realizar correcciones y permitir el mantenimiento futuro.

Dirigida a: personal técnico en informática (analistas y programadores).

Contiene: diseño de la aplicación, codificación de los programas y pruebas realizadas.

GU

Guía de Uso

Objetivo: dar a los usuarios finales toda la información necesaria para utilizar la aplicación.

Dirigida a: los usuarios que van a usar la aplicación (clientes).

Contiene: funcionalidad, forma de ejecutarla, ejemplos de uso, requerimientos software y solución de problemas.

GI

Guía de Instalación

Objetivo: garantizar que la implantación se realice de forma segura, confiable y precisa.

Dirigida a: personal informático responsable de la instalación, con los clientes.

Contiene: información para la puesta en marcha, explotación y seguridad del sistema.

Una correcta documentación permitirá la reutilización de parte de los programas en otras aplicaciones.

Explotación

Es la fase en que los usuarios finales conocen la aplicación y comienzan a utilizarla

1

Instalación

Los programas son transferidos al computador del usuario cliente y posteriormente configurados y verificados. Es recomendable que los clientes estén presentes.

2

Configuración

Se asignan los parámetros de funcionamiento normal de la empresa y se prueba que la aplicación es operativa. Puede hacerla el propio cliente o programarse de forma automática.

3

Producción normal

La aplicación pasa a manos de los usuarios finales y se da comienzo a la explotación del software.

En este momento se llevan a cabo las Beta Test, en los propios equipos del cliente y bajo cargas normales de trabajo.

Mantenimiento

La etapa más larga de todo el ciclo de vida del software

Por su naturaleza, el software es cambiante y deberá actualizarse y evolucionar con el tiempo. Se pacta con el cliente un servicio de mantenimiento, que también tendrá un coste temporal y económico.

Perfectivos

Para mejorar la funcionalidad del software

Evolutivos

El cliente tendrá nuevas necesidades: modificaciones, expansiones o eliminaciones de código

Adaptativos

Para adaptarse a las nuevas tendencias del mercado, a nuevos componentes hardware, etc.

Correctivos

La aplicación tendrá errores en el futuro (sería utópico pensar lo contrario)

Cuatro tipos de cambios que hacen necesario el mantenimiento.

El ciclo de vida del software

La serie de pasos a seguir para desarrollar un programa

Siempre se debe aplicar un modelo de ciclo de vida al desarrollo de cualquier proyecto software. Diversos autores han planteado distintos modelos; éstos son los más conocidos y utilizados.

Modelo en cascada

El clásico. Las etapas pasan de una a otra sin retorno posible

Cascada con realimentación

Uno de los más utilizados. Permite volver atrás entre etapas

Iterativo incremental

Evolutivo. Las fases se repiten y refinan

Modelo en espiral

Evolutivo. Versiones cada vez mejores, en ciclos

Los dos últimos se agrupan bajo el nombre de modelos evolutivos.

Modelo en cascada

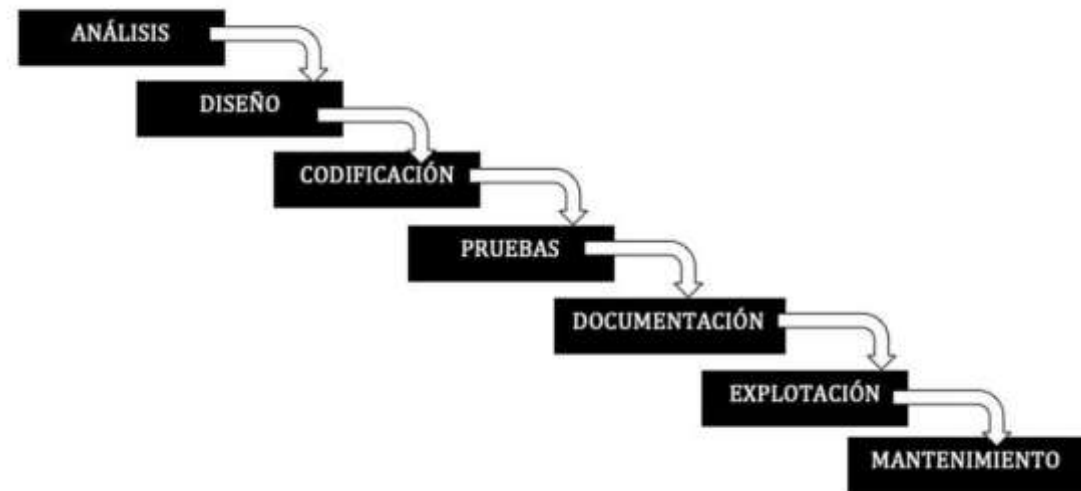
Es el modelo de ciclo de vida clásico del software y el de mayor antigüedad.

Requiere conocer de antemano todos los requisitos del sistema, por lo que es prácticamente imposible que se pueda utilizar.

Sólo es aplicable a pequeños desarrollos: las etapas pasan de una a otra sin retorno posible.

Las fases han de realizarse en el orden indicado. El resultado de una fase es la entrada de la siguiente.

Es un modelo bastante rígido que se adapta mal al cambio continuo de especificaciones.



Se presupone que no habrá errores ni variaciones del software.

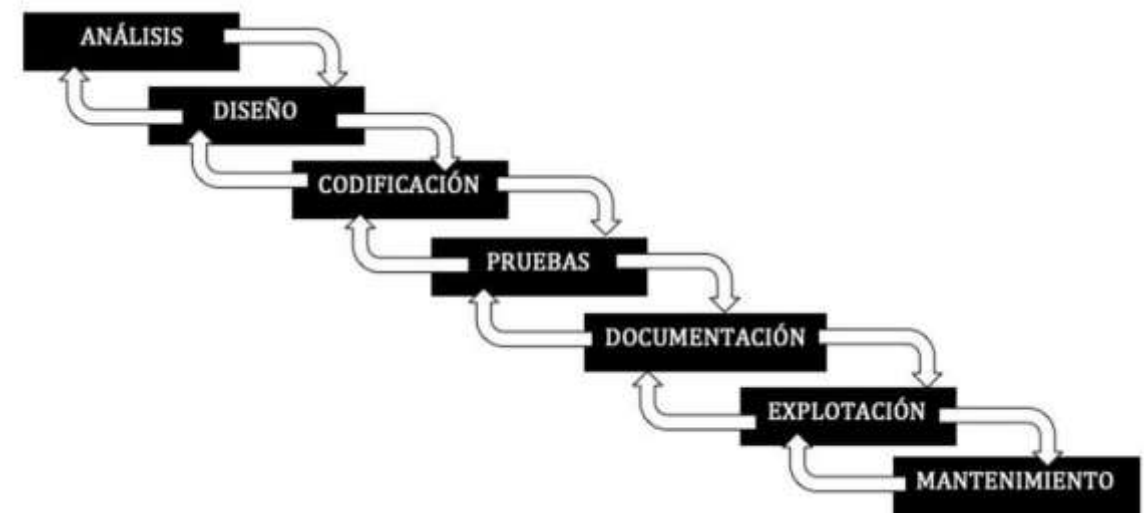
Modelo en cascada con realimentación

Es uno de los modelos más utilizados.

Proviene del modelo anterior, pero se introduce una realimentación entre etapas, de forma que podamos volver atrás en cualquier momento para corregir, modificar o depurar algún aspecto.

Si se prevén muchos cambios durante el desarrollo no es el modelo más idóneo.

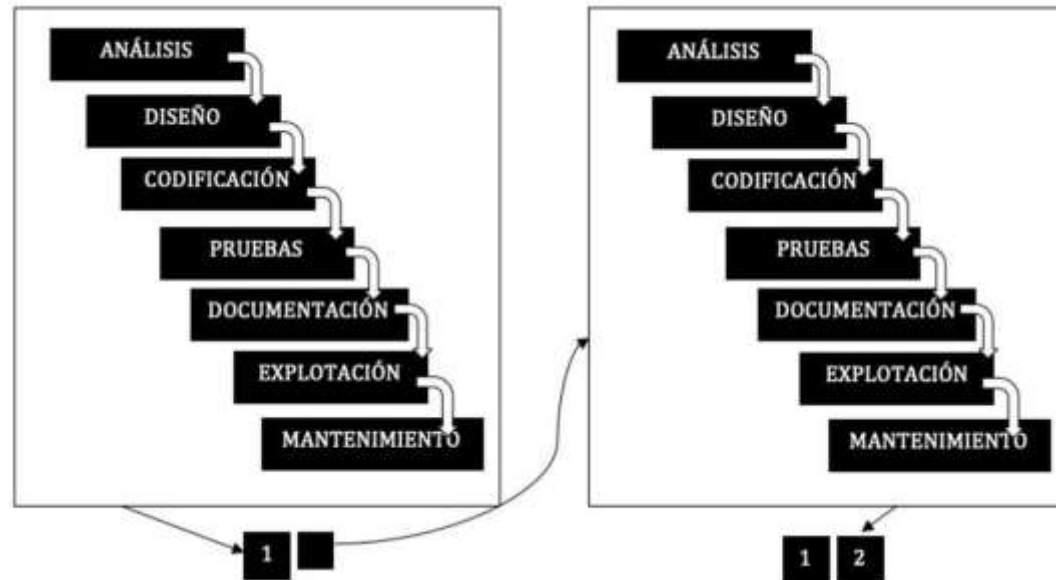
Es el modelo perfecto si el proyecto es rígido (pocos cambios, poco evolutivo) y los requisitos están claros.



La diferencia con el modelo anterior son las flechas de vuelta atrás entre etapas.

Modelo iterativo incremental

Los modelos evolutivos son más modernos que los anteriores: tienen en cuenta la naturaleza cambiante y evolutiva del software. Este está basado en el modelo en cascada con realimentación, donde las fases se repiten y refinan, y van propagando su mejora a las fases siguientes.



Cada iteración completa las siete fases y produce un incremento sobre la anterior.

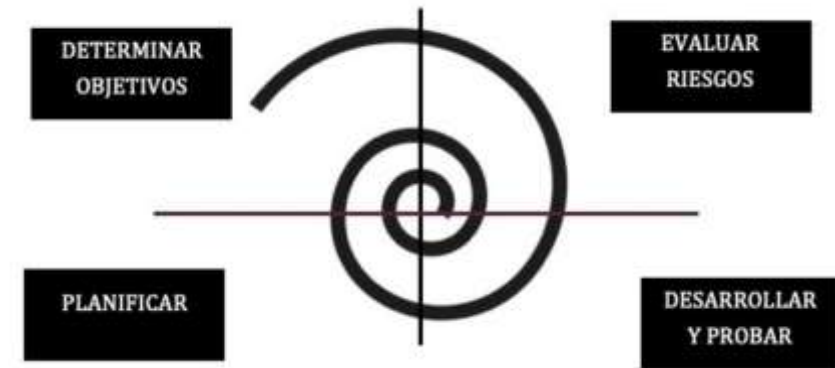
Modelo en espiral

Es una combinación del modelo anterior con el modelo en cascada.

El software se va construyendo repetidamente en forma de versiones que son cada vez mejores, debido a que incrementan la funcionalidad en cada versión.

Es un modelo bastante complejo.

Las actividades se conforman en una espiral: cada ciclo representa un conjunto de actividades.



Determinar objetivos

Evaluar riesgos

Desarrollar y probar

Planificar

Los cuatro modelos, cara a cara

Modelo	Rasgo característico	Cuándo conviene
En cascada	Las etapas pasan de una a otra sin retorno posible	Pequeños desarrollos en los que se conocen de antemano todos los requisitos
Cascada con realimentación	Realimentación entre etapas: se puede volver atrás para corregir o depurar	Proyectos rígidos, con pocos cambios y requisitos claros
Iterativo incremental	Las fases se repiten y refinan, propagando su mejora a las siguientes	Cuando se asume la naturaleza cambiante y evolutiva del software
En espiral	Versiones sucesivas que incrementan la funcionalidad en cada una	Combina los dos anteriores; es un modelo bastante complejo

Tabla elaborada con las características que el material atribuye a cada modelo.

¿Qué es un lenguaje de programación?

Idioma creado de forma artificial, formado por un conjunto de símbolos y normas que se aplican sobre un alfabeto para obtener un código que el hardware de la computadora pueda entender y ejecutar

Nos comunican con el hardware

Son los instrumentos que tenemos para que el ordenador realice las tareas que necesitamos

Hay multitud de lenguajes

Cada uno con unos símbolos y unas estructuras diferentes

Cada lenguaje tiene su terreno

Está enfocado a la programación de tareas o áreas determinadas

Por ello, la elección del lenguaje a utilizar en un proyecto es una cuestión de extrema importancia.

Los lenguajes también han evolucionado



La evolución va siempre hacia una mayor usabilidad: que el mayor número posible de usuarios los utilice.

Lenguaje máquina y lenguaje ensamblador

Lenguaje máquina

Sus instrucciones son combinaciones de unos y ceros.

Es el único lenguaje que entiende directamente el ordenador: no necesita traducción.

Fue el primer lenguaje utilizado.

Es único para cada procesador: no es portable de un equipo a otro.

Hoy día nadie programa en este lenguaje.

Lenguaje ensamblador

Sustituyó al lenguaje máquina para facilitar la labor de programación.

En lugar de unos y ceros se programa usando mnemotécnicos (instrucciones complejas).

Necesita traducción al lenguaje máquina para poder ejecutarse.

Sus instrucciones hacen referencia a la ubicación física de los archivos en el equipo.

Es difícil de utilizar.

Lenguajes de alto nivel y lenguajes visuales

De alto nivel basados en código

Sustituyeron al lenguaje ensamblador para facilitar más la labor de programación.

En lugar de mnemotécnicos se utilizan sentencias y órdenes derivadas del idioma inglés.

Necesitan traducción al lenguaje máquina.

Son más cercanos al razonamiento humano.

Son utilizados hoy día, aunque la tendencia es que cada vez menos.

Lenguajes visuales

Están sustituyendo a los lenguajes de alto nivel basados en código.

En lugar de sentencias escritas, se programa gráficamente usando el ratón y diseñando directamente la apariencia del software.

Su correspondiente código se genera automáticamente.

Necesitan traducción al lenguaje máquina.

Son completamente portables de un equipo a otro.

Alfabeto, sintaxis y semántica

Un lenguaje de programación es el conjunto de:

A

Alfabeto

Conjunto de símbolos permitidos

S

Sintaxis

Normas de construcción permitidas de los símbolos del lenguaje

S

Semántica

Significado de las construcciones para hacer acciones válidas

La elección del lenguaje para codificar un programa dependerá de las características del problema a resolver.

Cómo se clasifican los lenguajes

Según lo cerca que estén del lenguaje humano

De alto nivel: por su esencia, están más próximos al razonamiento humano.

De bajo nivel: están más próximos al funcionamiento interno de la computadora. Son el lenguaje ensamblador y el lenguaje máquina.

Según la técnica de programación utilizada

Estructurados: usan la técnica de programación estructurada. Ejemplos: Pascal, C, etc.

Orientados a objetos: usan la técnica de programación orientada a objetos. Ejemplos: C++, Java, Ada, Delphi, etc.

Visuales: basados en las técnicas anteriores, permiten programar gráficamente y el código se genera de forma automática. Ejemplos: Visual Basic.Net, Borland Delphi, etc.

Programación estructurada

Técnica para escribir lenguajes de programación que permite sólo el uso de tres tipos de sentencias o estructuras de control

Sentencias secuenciales

Las instrucciones se ejecutan una tras otra

Sentencias selectivas

Condicionales

Sentencias repetitivas

Iteraciones o bucles

Fue de gran éxito por su sencillez a la hora de construir y leer programas. Fue sustituida por la programación modular, que permitía dividir los programas grandes en trozos más pequeños, siguiendo la conocida técnica «divide y vencerás».

A su vez, luego triunfaron los lenguajes orientados a objetos y de ahí se pasó a la programación visual.

Estructurada: ventajas e inconvenientes

Ventajas

Los programas son fáciles de leer, sencillos y rápidos.

El mantenimiento de los programas es sencillo.

La estructura del programa es sencilla y clara.

Inconvenientes

Todo el programa se concentra en un único bloque: si se hace demasiado grande es difícil manejarlo.

No permite reutilización eficaz de código, ya que todo va «en uno».

Ejemplos de lenguajes estructurados: Pascal, C, Fortran

Programación orientada a objetos

Los programas no se tratan como un conjunto ordenado de instrucciones, sino como un conjunto de objetos independientes entre sí que colaboran para realizar acciones

Atributos

Los objetos del programa tendrán una serie de atributos que los diferencian unos de otros

Clases

Una clase es una colección de objetos con características similares

Métodos

Mediante los métodos los objetos se comunican con otros, produciéndose un cambio de estado

Objetos

Unidades individuales e indivisibles que forman la base de este tipo de programación

Los objetos son reutilizables para proyectos futuros.

Por qué se impuso la POO

1

El código es reutilizable

Los objetos pueden emplearse en proyectos futuros.

2

Los errores se localizan antes

Si hay algún error, es más fácil de localizar y depurar en un objeto que en un programa entero.

Su primera desventaja es clara: no es una programación tan intuitiva como la estructurada. Aun así, la mayoría del software que producen las empresas se hace usando esta técnica.

Principales lenguajes orientados a objetos: Ada, C++, VB.NET, Delphi, Java, PowerBuilder, etc.

Frameworks

Plataforma, entorno o marco de trabajo del desarrollo rápido de aplicaciones: una estructura de ayuda al programador en base a la cual podemos desarrollar proyectos sin partir de cero

Es una plataforma software donde están definidos programas soporte, bibliotecas, lenguaje interpretado, etc., que ayuda a desarrollar y unir los diferentes módulos o partes de un proyecto. Con un framework podemos pasar más tiempo analizando los requerimientos del sistema y las especificaciones técnicas de nuestra aplicación, ya que la tarea laboriosa de los detalles de programación queda resuelta.

.NET

Framework para desarrollar aplicaciones sobre Windows. Ofrece «Visual Studio .net» y su motor es el «.NET Framework». Se instala sobre el sistema operativo.

Spring de Java

Conjuntos de bibliotecas (API's) para el desarrollo y ejecución de aplicaciones.

Frameworks: ventajas e inconvenientes

Ventajas

Desarrollo rápido de software.

Reutilización de partes de código para otras aplicaciones.

Diseño uniforme del software.

Portabilidad de aplicaciones de un computador a otro.

Inconvenientes

Gran dependencia del código respecto al framework utilizado: si cambiamos de framework, habrá que reescribir gran parte de la aplicación.

La instalación e implementación del framework en nuestro equipo consume bastantes recursos del sistema.

La portabilidad que aporta el framework se suma a la que ya proporciona la máquina virtual.

Entornos de ejecución

Servicio de máquina virtual que sirve como base software para la ejecución de programas: un conjunto de utilidades que permiten ejecutar programas

En ocasiones pertenece al propio sistema operativo, pero también se puede instalar como software independiente que funcionará por debajo de la aplicación. Durante la ejecución, los entornos se encargarán de:

Configurar la memoria

Configurar la memoria principal disponible en el sistema

Enlazar

Enlazar los archivos del programa con las bibliotecas existentes y con los subprogramas creados

Depurar

Comprobar la existencia o no existencia de errores semánticos del lenguaje

Los errores sintácticos ya se detectaron en la compilación.

De qué está formado y qué le falta

Máquina virtual



API's: bibliotecas de clase estándar

Necesarias para que la aplicación, escrita en algún Lenguaje de Programación, pueda ser ejecutada.

Estos dos componentes se suelen distribuir conjuntamente, porque necesitan ser compatibles entre sí. El entorno funciona como intermediario entre el lenguaje fuente y el sistema operativo, y consigue ejecutar aplicaciones.

Pero si lo que queremos es desarrollar nuevas aplicaciones, no es suficiente con el entorno de ejecución: hace falta un ENTORNO DE DESARROLLO

Ese «algo más» es el contenido de la próxima unidad.

EN UNA DIAPOSITIVA

Lo que hay que retener

- 1 Hay tres tipos de software: de sistema, de desarrollo y de aplicación.
- 2 Desarrollar software es todo el proceso desde la idea hasta el programa funcionando.
- 3 Las siete fases: análisis, diseño, codificación, pruebas, documentación, explotación y mantenimiento.
- 4 El análisis es la fase más importante y culmina en el documento ERS.
- 5 El código pasa por tres estados: fuente, objeto y ejecutable.
- 6 Compilar traduce de una sola vez; interpretar traduce y ejecuta línea a línea y no produce código objeto.
- 7 El mantenimiento es la etapa más larga: perfectivo, evolutivo, adaptativo y correctivo.
- 8 Modelos de ciclo de vida: cascada, cascada con realimentación y evolutivos (iterativo incremental y espiral).
- 9 Un lenguaje es alfabeto + sintaxis + semántica, y se clasifica por nivel y por técnica.
- 10 El entorno de ejecución es máquina virtual + API's; para desarrollar hace falta un entorno de desarrollo.

